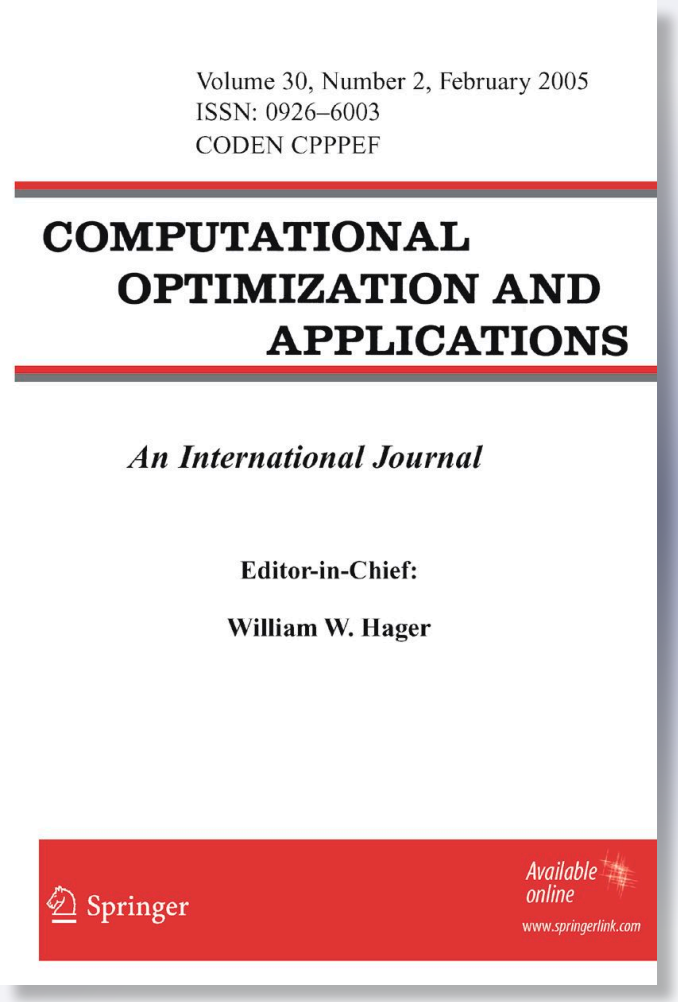
Kernel Search: a new heuristic framework for portfolio selection

Enrico Angelelli, Renata Mansini & M. Grazia Speranza

Computational Optimization and Applications
An International Journal

ISSN 0926-6003
Volume 51
Number 1

Comput Optim Appl (2012) 51:345-361
DOI 10.1007/s10589-010-9326-6



Springer

Your article is protected by copyright and all rights are held exclusively by Springer Science+Business Media, LLC. This e-offprint is for personal use only and shall not be self-archived in electronic repositories. If you wish to self-archive your work, please use the accepted author's version for posting to your own website or your institution's repository. You may further deposit the accepted author's version on a funder's repository at a funder's request, provided it is not made publicly available until 12 months after publication.

Springer

Author's personal copy

Comput Optim Appl (2012) 51:345–361
DOI 10.1007/s10589-010-9326-6

Kernel Search: a new heuristic framework for portfolio selection

Enrico Angelelli · Renata Mansini · M. Grazia Speranza

Received: 3 March 2009 / Published online: 18 March 2010
© Springer Science+Business Media, LLC 2010

Abstract In this paper we propose a new heuristic framework, called Kernel Search, to solve the complex problem of portfolio selection with real features. The method is based on the identification of a restricted set of promising securities (*kernel*) and on the exact solution of the MILP problem on this set. The continuous relaxation of the problem solved on the complete set of available securities is used to identify the initial kernel and a sequence of integer problems are then solved to identify further securities to insert into the kernel. We analyze the behavior of several heuristic algorithms as implementations of the Kernel Search framework for the solution of the analyzed problem. The proposed heuristics are very effective and quite efficient. The Kernel Search has the advantage of being general and thus easily applicable to a variety of combinatorial problems.

Keywords Kernel Search · Portfolio optimization with real features · Mixed Integer Linear Programming

1 Introduction

In portfolio selection problems the investor aims at selecting a set of securities to optimize a performance measure under specific constraints. If, from a theoretical point of view, the portfolio optimization is one of the most challenging and studied financial problems, the recent dramatic fall experienced by financial markets has fueled a new interest in the problem also from a practical point of view.

E. Angelelli · M.G. Speranza (✉)
Department of Quantitative Methods, University of Brescia, Brescia, Italy
e-mail: speranza@eco.unibs.it

R. Mansini
Department of Information Engineering, University of Brescia, Brescia, Italy

Author's personal copy

In the last years the research community has made significant advances in this area. On one side the research has been devoted to provide evidence on how effective the mathematical models are as decision tools (see, the work by Markowitz [17], milestone in theoretical portfolio analysis and, more recently, works by Konno and Yamamoto [12] and Guastaroba et al. [7], where the models effectiveness is shown through experimental studies). On the other side, the focus has been put on the identification of efficient and effective solution algorithms for portfolio optimization problems where the inclusion of practical features has implied the construction of models with binary and integer variables. Even if some attempts have been made in developing exact algorithms (see Konno and Wijayanayake [10] and Mansini and Speranza [14]), the design of effective and efficient heuristic methods has been considered as the most critical issue in portfolio optimization problems.

The literature offers a large variety of heuristics and metaheuristics for portfolio selection problems (see, for instance, Chang et al. [4], Jobst et al. [8] and Mansini and Speranza [13] and more recently Konno and Yamamoto [11]). The heuristics tend to be problem-dependent and, thus, not easily extendable to other, even if very similar, problems. Typically, a little change in the problem for which they were designed makes them useless and implies the design of new heuristics, or at least deep changes. This lack of robustness is indeed one of the major causes for the difficulties occurring in the practical use of heuristics for most combinatorial problems.

In this paper we propose a general heuristic framework for portfolio optimization problems which we call *Kernel Search*. The main idea of the method is to obtain a solution, of hopefully high quality, from a small set of promising securities, which we call the *kernel*. The kernel is initially built using information provided by the solution of the linear relaxation of the original problem. Then, new promising securities are identified by solving a sequence of small/moderate size MILP problems. The first MILP problem in the sequence is restricted to the initial kernel. Any other MILP problem in the sequence is restricted to the previous kernel plus a set of other securities that were initially excluded. The solution of the current MILP problem may identify some new securities not yet included in the kernel. If this is the case, such new securities are added to the kernel. The possibly updated kernel will be forwarded to the next MILP problem of the sequence. The kernel increases in a monotonic way, i.e. no security will be discharged at any time, and the solution of any MILP problem in the sequence provides a bound on the optimal solution for all the successive ones. One of the main issues we address concerns the size of these MILP problems. This value should be small enough to limit the computational time required to solve each MILP problem and large enough to be likely to contain most of the difficult to select securities (i.e. those that can be selected only if all securities were considered altogether).

Different heuristics can be designed as implementations of the proposed Kernel Search framework. Such heuristics have two major characteristics relevant from a practical point of view. The first one is that they require little implementation effort because the most cumbersome part of the search is carried out by a commercial software for the solution of linear and mixed integer linear programming problems. The second characteristic is that the same heuristic can be easily applicable also to other problems. In this paper we analyze several of such heuristics and test them on a complex portfolio optimization problem taking different real features such as minimum

Author's personal copy

transaction lots and cardinality constraint into account. The model maximizes a performance measure represented by the Worst Conditional Expectation (see Mansini et al. [15]). The Kernel Search framework exploits a major characteristic of the portfolio selection problem which is the fact that the number of securities selected by an optimization model is usually quite small, independently of the initial size of the problem. For this reason, any other mixed integer linear programming formulation using a different performance measure would have played in this paper the same role of the problem we have analyzed.

The selected portfolio problem based on Worst Conditional Expectation has never been solved in the literature. The effectiveness and efficiency of the proposed heuristic framework is thus studied by comparing its performance to that of CPLEX, one of the best commercial software for solving mixed integer linear programming problems. CPLEX exact routines and our heuristic framework effectiveness are analyzed by comparing the results obtained by the two procedures within a defined threshold time.

In the literature other heuristics can be found that are based on the selection of a restricted set of the available items in order to reduce the analysis of the whole solution space to a promising region. Examples of such approaches can be found for Knapsack Problems (see the core algorithm proposed by Balas and Zemel [3] and developed by Pisinger [18]) and in the context of routing problems (see Archetti et al. [2] and De Franceschi et al. [6]), where mixed integer linear programming models are solved to deeply explore a promising subset of the solution space. However, such effective heuristics are much more problem-dependent than the Kernel Search we present in this paper.

The paper is organized as follows. In Sect. 2 we provide the mathematical formulation of the analyzed portfolio optimization problem with real features. Section 3 introduces the Kernel Search framework while Sect. 4 is devoted to the description of different heuristics obtained as implementations of the Kernel Search to solve this problem. Section 5 provides the computational analysis. Finally, some conclusions are drawn and future developments are sketched.

2 The portfolio optimization problem

Markowitz [16, 17] first formalized the portfolio optimization problem as a bi-criteria optimization problem where the risk, measured as variance, is minimized and the expected return is maximized. Recently, new linear portfolio selection problems based on performance measures alternative to the variance have been proposed (see Mansini et al. [15] for a theoretical analysis of these Linear Programming (LP) solvable models and their computational comparison on real data).

In this paper, we consider a single-period portfolio optimization problem with real features modeling the problem of an investor who wants to allocate a capital on a set $N = \{1, \ldots, n\}$ of securities by assigning a nonnegative weight (share of capital) to each of them while optimizing a performance measure at a minimum required return. The objective is the maximization of the Worst Conditional Expectation consisting in the portfolio mean return under a specified β-quantile of worst realizations.

Author's personal copy

Portfolio real features are necessary or desired conditions on the investment which require binary and integer variables to be explicitly modeled. Different real features can be taken into account. In [13], Mansini and Speranza have analyzed the portfolio optimization problem with minimum transaction lots and proposed some heuristics based upon the idea of constructing and solving mixed integer subproblems on a subset of securities selected among the most interesting securities identified by the optimal solution of the continuous relaxation problem. Since the proposed heuristics were quite general, the authors applied them to different mixed integer portfolio models (see Chiodi et al. [5] and Kellerer et al. [9]). Starting from the insights provided by these results, Angelelli et al. [1] have analyzed and compared two MILP models for portfolio optimization taking into account integer units, transaction costs and a cardinality constraint. The authors have shown that, while from a theoretical point of view the continuous relaxation problem does not necessarily select all the securities contained in the optimal integer solution, from a computational point of view there is evidence that the subset of securities selected by the continuous relaxation problem is highly correlated with the set of optimal securities.

In the proposed problem the selected portfolio has to meet the following practical conditions (real features): the investor pays a fixed cost f_j and a proportional cost c_j for any selected security $j \in N$; each security can be acquired only in multiples of a transaction unit (round) and a minimum l_j and a maximum u_j threshold on the number of rounds invested in each selected security as well as a maximum number of securities m held in the portfolio are also considered. Two sets of variables are taken into account to correctly model the problem. Variables x_j denote the number of rounds selected for each security j, $j \in N$, and each binary variable z_j takes value 1 when the corresponding security is selected in the portfolio and 0 otherwise. We indicate as q_j the quotation of security j at the date of portfolio optimization so that $q_j x_j$ is the portfolio weight (share) invested in security j. When investing in security j, the general Stock Exchange practice imposes the payment of a minimum amount K_j of proportional transaction costs. In order to correctly formalize such situation we have assumed that, when investing in a security j, the investor will always buy a number of rounds x_j such that $c_j q_j x_j \geq K_j$. Since variables x_j are integer, this is equivalent to set a lower bound $l_j = \lceil \frac{K_j}{c_j q_j} \rceil$ for each security $j \in N$. Let C_0 and C_1 be the lower and the upper bound on the total capital available for the portfolio investment. We assume that the amount $q_j x_j$ invested in each security j cannot exceed a fixed percentage α of C_0, with $0 < \alpha < 1$. Thus, the following upper bound $u_j = \lfloor \alpha \frac{C_0}{q_j} \rfloor$ applies to the maximum number of units for security $j \in N$.

We consider T scenarios with probabilities p_t, where $t = 1, \ldots, T$. Since scenarios are represented by historical data we assume they are all equally probable, i.e. $p_t = 1/T$, $t = 1, \ldots, T$. For each security $j \in N$ we define as R_j the random variable representing its rate of return and as r_{jt} its realization under the specified scenario t. The portfolio has mean return $r_j = E\{R_j\} = \sum_{t=1}^{T} r_{jt}/T$. The portfolio optimization problem with real features can be formulated as follows:

$$\textit{IP-Model}(N) \quad w := \max\left(y - \frac{1}{\beta} \sum_{t=1}^{T} p_t d_t \right) \tag{1}$$

Author's personal copy

$$y - \sum_{j=1}^{n}(r_{jt} - c_j)q_j x_j + \sum_{j=1}^{n} f_j z_j \leq d_t \quad t = 1, \ldots, T \tag{2}$$

$$\sum_{j=1}^{n}(r_j - c_j)q_j x_j - \sum_{j=1}^{n} f_j z_j \geq \mu_0 \sum_{j=1}^{n} q_j x_j \tag{3}$$

$$\sum_{j=1}^{n} z_j \leq m \tag{4}$$

$$l_j z_j \leq x_j \leq u_j z_j \quad j = 1, \ldots, n \tag{5}$$

$$C_0 \leq \sum_{j=1}^{n} q_j x_j \leq C_1 \tag{6}$$

$$d_t \geq 0 \quad t = 1, \ldots, T \tag{7}$$

$$x_j \geq 0 \quad \text{integer}, \quad j = 1, \ldots, n \tag{8}$$

$$z_j \in \{0, 1\} \quad j = 1, \ldots, n. \tag{9}$$

The objective function (1), together with constraints (2) and (7), models the Worst Conditional Expectation. The parameter β represents the specified size (quantile) of worst realizations and the objective function measures the portfolio mean return under the k worst scenarios, where $k = \beta T$. The variable y represents an independent free variable that at optimality takes the value of the β-quantile. Constraints (2) along with constraints (7) define each variable d_t as the $\max\{0, y - y_t\}$, where $y_t = \sum_{j=1}^{n}(r_{jt} - c_j)q_j x_j - \sum_{j=1}^{n} f_j z_j$ is the net return portfolio realization under scenario t. Constraint (3) imposes that the portfolio net mean return is greater than or equal to the required return μ_0. Constraint (4) expresses the cardinality condition by imposing that the number of securities selected in the portfolio has to be not greater than m. Constraint (6) says that the capital invested in the portfolio must be included between the lower and the upper bounds C_0 and C_1, respectively. If a security j is selected in the portfolio, i.e. $x_j > 0$, then constraint (5) forces the number of rounds to be included between l_j and u_j. Finally, constraints (7)–(9) are nonnegativity and integrality conditions.

We indicate as *LP-Model*(N) the corresponding continuous relaxation problem, obtained by substituting constraints (8)–(9) with their continuous relaxation:

$$x_j \geq 0 \quad j = 1, \ldots, n \tag{10}$$

$$0 \leq z_j \leq 1 \quad j = 1, \ldots, n. \tag{11}$$

3 Kernel Search framework

The Kernel Search framework is designed upon the idea to select a kernel Λ of promising securities using the information provided by the optimal solution of the initial continuous relaxation problem and then optimally solve the original mixed

Author's personal copy

integer programming problem by only considering the securities in Λ (problem *IP-Model*(Λ)). The set Λ should be small enough to allow us to find the optimal solution of *IP-Model*(Λ) within a reasonable amount of computational time (solution efficiency) and large enough to likely include the securities that make part of the optimal solution of *IP-Model*(N) (solution quality). Then, the kernel is possibly increased through a refined search that involves the solution of several problems of the kind *IP-Model*(Λ') where $\Lambda' \supset \Lambda$. Parameter C denotes the number of securities that form the initial kernel.

Basic Kernel Search

INPUT: Set N of available securities.
OUTPUT: A feasible solution $(\mathbf{x}^*, \mathbf{z}^*)$ and its value w^* if available, otherwise $w^* := -\infty$.

Initialization Phase (*construction of the initial kernel* Λ *and sequence of buckets*)

1. Solve the continuous relaxation problem *LP-Model*(N).
2. Sort the securities according to a predefined sorting criterion.
3. Consider the securities as sorted in step 2:
   - Construct the initial kernel Λ by selecting the first C securities;
   - Use the set $N \setminus \Lambda$ to construct a sequence $\{B_i\}$ of buckets;
   - Let NB be the number of generated buckets.
4. Solve *IP-Model*(Λ). Let $(\mathbf{x}^*, \mathbf{z}^*)$ and w^* be the optimal solution and its value, if available. Otherwise, set $w^* := -\infty$.

Extension Phase (*extension of the kernel* Λ)

1. Set $i := 1$.
2. Construct the set $\Lambda_i := \Lambda \cup B_i$ (B_i is the current bucket).
3. Solve *IP-Model*(Λ_i) by adding the two constraints:
   (a) $\sum_{j \in B_i} z_j \geq 1$;
   (b) Set w^* as lower bound to the problem objective function value.
4. **if** *IP-Model*(Λ_i) is feasible **then**
   - Let $(\bar{\mathbf{x}}, \bar{\mathbf{z}})$ be its optimal solution with value $\bar{w}$;
   - Set $w^* := \bar{w}$, $\mathbf{x}^* := \bar{\mathbf{x}}$, $\mathbf{z}^* := \bar{\mathbf{z}}$;
   - Define $\bar{\Lambda}_i$ as the set of securities belonging to B_i that have been selected in $(\bar{\mathbf{x}}, \bar{\mathbf{z}})$;
   - Set $\Lambda := \Lambda \cup \bar{\Lambda}_i$.
5. **if** $i < NB$ **then**
   - Set $i := i + 1$;
   - Go to step 2 of the current phase;
   otherwise STOP.

The basic framework consists of two main phases. The *Initialization Phase* is mainly devoted to the construction of an initial kernel and of a sequence of buckets.

Author's personal copy

In Step 1 the continuous relaxation of the original problem on the whole set N of available securities (*LP-Model*(N)) is solved. In Step 2 the securities are sorted according to a predefined criterion, by using the information provided by the continuous solution. Different rules can be used as sorting criteria. The idea is to find a criterion such that the securities which are more likely to belong to the optimal integer solution come first. In Step 3, the kernel Λ is initialized by selecting the first C securities of this ordered set, where C is a given parameter, while the remaining securities are divided into groups called *buckets*. Different implementations of the framework may generate sequences of buckets $\{B_i\}$ that differ for the number NB of buckets and for their cardinality (*bucket length*). Finally, a first *IP-Model*(Λ) is solved in order to obtain a lower bound for the problem. In Step 4 of the *Initialization Phase*, if the problem *IP-Model*(Λ) cannot find a feasible solution, the initial value of the lower bound w^* is set to $-\infty$.

The *Extension Phase* is devoted to the enlargement of the initial kernel Λ by analyzing the constructed buckets one after the other. At each iteration the securities in the current bucket B_i are temporarily added to the kernel Λ providing a new set $\Lambda_i := \Lambda \cup B_i$. The integer subproblem *IP-Model*(Λ_i) is then solved with the addition of two constraints (see Step 4 of this phase) to reduce the computational time. Indeed, we are interested only in those solutions that improve on the previous lower bound by using at least one new item from the current bucket. If *IP-Model*(Λ_i) has a feasible solution and the solver is able to find a feasible solution (possibly the optimal one), the procedure looks for the securities selected from the current bucket B_i (set Λ_i). Such securities are permanently added to the kernel $\Lambda := \Lambda \cup \bar{\Lambda}_i$. The procedure is then iterated on a new bucket. When the last bucket has been analyzed the procedure stops by providing $(\mathbf{x}^*, \mathbf{z}^*)$ with value w^* as best found solution.

The described approach takes into account the generated sequence of buckets only once and is referred as the basic variant (*Basic Kernel Search*). We also propose an *Iterative Kernel Search* where the *Extension Phase* is executed more than once. In this variant, as soon as the current sequence of the buckets has been completely analyzed, the *Extension Phase* is restarted, provided that some new securities have been added to the kernel. The new *Extension Phase* is initialized by setting $NB := q - 1$, where $q := \max\{i \,|\, \bar{\Lambda}_i \neq \emptyset\}$ is the index of the last bucket from which at least one item has been selected to enter the kernel Λ. If a new item is selected, then NB is set to its original value as evaluated at step 3 of the initialization phase. The rationale for this variant is that the inclusion into the kernel of new securities from a bucket may change the desirability of some other securities which belong to already analyzed buckets and that, at present, have not been selected. The Iterative Kernel Search stops when no new item is added to set Λ, i.e. $\bar{\Lambda}_i = \emptyset$ for all $i \leq NB$.

4 Kernel Search heuristics

The Kernel Search framework generates possibly different heuristic implementations according to the criterion used to sort the items in step 2 of the initialization phase, the number of items initially selected to construct the kernel (parameter C), the type of buckets taken into account (disjoint or overlapping, with fixed or variable length) and

Author's personal copy

some parameters that define more specifically how to build the sequence of buckets, such as the number of buckets to consider (parameter *NB*) and the number of items making part of each bucket (parameter *lbuck*). Finally, the Basic or Iterative version has to be selected.

In the following we present the choice we made for each parametric feature of the framework. Most of them are absolutely independent from the specific optimization model, others, like one of the adopted sorting criteria, take into account the characteristics of the portfolio model. Nevertheless, we believe that such criterion is quite general and can be easily applied to other problems or substituted by an analogous model-dependent criterion.

Sorting criterion The amount $q_j x_j$ is the share of security j and the quantity $\hat{c}_j$ is the reduced cost of variable x_j in the optimal solution of *LP-Model*(N). We have considered three sorting criteria based on the non-increasing order of shares and, for null shares, the non-increasing order of the values $\hat{c}_j l_j$ (Sorting A), of the reduced costs $\hat{c}_j$ (Sorting B) or a random order (Sorting R).

The rationale for these three sorting criteria is to progressively take into account information from the solution of the linear relaxation of the problem. The last criterion considers as interesting only those securities included in the *LP* relaxation of the problem, all other securities are considered at the same level. The second criterion makes one step further and among those securities excluded from the *LP* relaxation, considers as more interesting those that have a smaller (negative) reduced cost. The idea here is that reduced costs represent the sensitivity of the objective function to the introduction of a new security into the solution. We use the reduced cost of variables x_j and not those of the binary variables z_j, because the former are the variables more strongly influencing the objective function value. Finally, the first criterion tries to exploit a specific feature of the problem. Since any security j can enter the solution only at a minimum level of the corresponding x variable ($x_j \geq l_j$), then a variable x_j out of the basis will decrease the objective function value of at least $\hat{c}_j l_j$ if selected. We consider as more interesting those securities that imply a lower reduction of the objective function value when entering the solution.

Kernel initialization For all the tested heuristics, the parameter C has been set equal to the number of securities selected in the optimal solution of problem *LP-Model*(N).

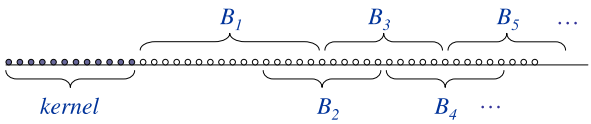
Number and structure of the buckets The number of buckets and their size as well as the rule used to construct them can vary significantly. We have identified the following possibilities:

- *Fixed* versus *Variable* bucket length. Buckets may have the same fixed length or a length that changes according to some predefined rule.

  We define as *Fixed-bucket*(k, NB) the heuristics based on the generation of disjoint buckets all with the same predefined length $lbuck$. The first parameter k refers to the fact that we have assumed the fixed length of each bucket equal to a multiple k of the cardinality of the initial kernel Λ, i.e. $lbuck = kC$. The second parameter *NB* represents the number of buckets. This can be set a priori or it may come out as a result of the buckets construction. In the latter case the parameter is omitted

Author's personal copy

Fig. 1 Shifted-bucket heuristic



and we identify the heuristic simply as *Fixed-bucket*(k). All the generated buckets will have cardinality *lbuck*, but possibly the last one that may contain a number of securities less than *lbuck*. In the first seven lines of Table 1 the *Fixed-bucket* heuristics we have decided to implement are listed and briefly described.

We define as *Variable-bucket*(**s**), where **s** is a vector of parameters, the heuristics based on the generation of a sequence of disjoint buckets with variable (decreasing/increasing) length. Although more complex functions may be considered, after some preliminary tests, we concentrated our attention on a non-increasing piecewise linear function where the length of each bucket B_i, $i \geq 1$, is computed as $\max\{s_1, C - s_2(i-1)\}$ where s_1 is the minimum bucket length while s_2 is the slope of the linear relation. Vector **s** is the set of parameters of the linear function. Since we consider the function with $s_1 = 5$ and $s_2 = 10$, we identify this heuristic as *Variable-bucket*(5, 10).

- *Overlapping* versus *Disjoint* buckets. Generated buckets may be disjoint sets or may overlap according to some rule. *Fixed-bucket* and *Variable-bucket* heuristics are examples of disjoint buckets. In the case of overlapping buckets we have generated two sequences of buckets, each one characterized by a predefined length of the buckets and by a given starting position of the first security. The two sequences are then merged into a single one with the following rule: the buckets that contain more promising securities according to the criterion used to sort them come first.

  Several heuristics can be worked out according to the type of buckets generated for each sequence and the rule used to sort the merged sequence. After several experiments, we have decided to report the results for only one heuristic identified as *Shifted-bucket*. In this procedure all the buckets have cardinality equal to C, but for the first one that has length $1.5C$. Moreover, the first security of the first sequence is in position $C + 1$ while the first bucket in the second sequence starts in position $2C + 1$. Figure 1 provides a graphical description of the bucket sequence generated by *Shifted-bucket*.

Basic and iterative variants For each heuristic characterized by a number of buckets greater than one, we have implemented both the basic framework and its iterative variant.

The complete set of tested heuristics is listed and briefly described in Table 1. All heuristics have been implemented and tested with all three sorting criteria.

5 Experimental analysis

The analyzed heuristics have been implemented in Visual C++ and the integer problems have been solved by means of CPLEX 9.0. Experimental testing has been conducted on two main data sets by means of a PC AMD Athlon(TM) XP 2000+ with a 3 GHz Pentium processor and 2 GB of RAM.

Author's personal copy

Table 1 All tested heuristics

Basic Kernel Search heuristics:	
Fixed-bucket(0,0)	$k = 0, NB = 0$; kernel = items with positive value in LP optimal solution.
Fixed-bucket(1,1)	$k = 1, NB = 1$; one bucket with cardinality C.
Fixed-bucket(2,1)	$k = 2, NB = 1$; one bucket with cardinality $2C$.
Fixed-bucket(1,2)	$k = 1, NB = 2$; two buckets with fixed cardinality equal to C.
Fixed-bucket(0.2,10)	$k = 0.2, NB = 10$; ten buckets with fixed cardinality $\frac{1}{5}C$.
Fixed-bucket(0.2)	$k = 0.2$; items are divided into buckets of fixed length $\frac{1}{5}C$.
Fixed-bucket(1)	$k = 1$; items are divided into buckets of fixed length C.
Variable-bucket(5,10)	bucket $B_i, i \geq 1$, has length $\max\{5, C - 10(i-1)\}$.
Shifted-bucket	two sequences of overlapping buckets merged into one. First sequence: B_1 has length $1.5C$, the remaining ones C. Second sequence: all buckets have length C.
Iterative Kernel Search heuristics:	
Fixed-bucket-I(1,2)	iterative version of *Fixed-bucket*(1,2).
Fixed-bucket-I(0.2,10)	iterative version of *Fixed-bucket*(0.2,10).
Fixed-bucket-I(0.2)	iterative version of *Fixed-bucket*(0.2).
Fixed-bucket-I(1)	iterative version of *Fixed-bucket*(1).
Variable-bucket-I(5,10)	iterative version of *Variable-bucket*(5,10).

5.1 European data set

The main data set consists of 104 historical realizations representing weekly rates of return of 152 Italian, 224 French and 224 German securities over the period (1999–2000). The rates are computed as relative stock price variations. Dividends are not included. We have assumed to solve the portfolio optimization problem for an Italian investor who pays lower commissions when buying Italian stocks with respect to foreign ones. Fixed transaction costs have been set to 5 Euro for all the securities, i.e. $f_j = 5, \forall j$, while proportional costs c_j are equal to 0.7% for each Italian security and to 0.9% for each foreign (French or German) security. The fixed amount of transaction commission K_j, used to compute the lower bound on investment for each security, has been set to 12 Euro for the Italian market and to 50 Euro for the French and German ones. The capital ranges from $C_0 = 100000$ to $C_1 = C_0(1 + 1\%)$ Euro, while the minimum required return has been fixed to $\mu_0 = 5\%$. The parameter α used to determine the upper bound on the number of rounds selected for each security j in the portfolio has been set to 15%.

Different instances have been created according to the number of securities used ($n = 100$, 200, 300, 400, 500 and 600 securities, respectively), the threshold on portfolio cardinality ($m =$10 and 20) and the tolerance level ($\beta = 0.01$ and 0.05). This means 24 instances altogether. For each instance, we have tested all the 14 proposed heuristics (see Table 1) with respect to all the three described sorting criteria (A, B, R).

Since solving to optimality large size instances may require huge computational times, we have set to 12 hours the maximum running time assigned to the CPLEX optimizer. This means that, if the time limit is reached without finding the integer

Author's personal copy

Table 2 Computational times (in seconds) truncated to integer values: Optimal integer solutions

	$\beta = 1\%$ $m = 10$	$\beta = 5\%$ $m = 10$	$\beta = 1\%$ $m = 20$	$\beta = 5\%$ $m = 20$
100	3	6	1	0
200	142	112	21	15
300	8814	6515	100	91
400	10761	6798	162	143
500	(*)	(*)	1914	1352
600	(*)	(*)	18757	13023

Table 3 Percentage of instances solved within a given error γ from the optimal solution: Bucket-based heuristics

	Sorting A				Sorting B				Sorting R			
	0%	1%	5%	10%	0%	1%	5%	10%	0%	1%	5%	10%
Fixed-bucket(0,0)	0.25	0.38	0.63	0.83	0.25	0.38	0.63	0.83	0.25	0.38	0.63	0.83
Fixed-bucket(0.2,10)	0.54	0.67	1.00	1.00	0.50	0.67	1.00	1.00	0.29	0.54	0.79	0.92
Fixed-bucket(0.2)	0.58	0.75	1.00	1.00	0.71	0.79	1.00	1.00	0.46	0.71	1.00	1.00
Fixed-bucket(1,1)	0.71	0.79	1.00	1.00	0.29	0.58	0.83	1.00	0.25	0.54	0.88	0.92
Fixed-bucket(1,2)	0.75	0.92	1.00	1.00	0.58	0.75	1.00	1.00	0.29	0.54	0.88	0.92
Fixed-bucket(2,1)	0.75	0.92	1.00	1.00	0.71	0.88	1.00	1.00	0.29	0.54	0.88	0.96
Fixed-bucket(1)	0.79	1.00	1.00	1.00	0.67	0.83	1.00	1.00	0.46	0.79	1.00	1.00
Variable-bucket(5,10)	0.79	1.00	1.00	1.00	0.58	0.75	1.00	1.00	0.46	0.79	1.00	1.00
Shifted-bucket	0.79	1.00	1.00	1.00	0.71	0.83	1.00	1.00	0.46	0.79	1.00	1.00
Fixed-bucket-I(0.2,10)	0.63	0.75	1.00	1.00	0.54	0.71	1.00	1.00	0.29	0.54	0.79	0.92
Fixed-bucket-I(0.2)	0.71	0.83	1.00	1.00	0.79	0.83	1.00	1.00	0.71	0.83	1.00	1.00
Fixed-bucket-I(1,2)	0.75	0.92	1.00	1.00	0.67	0.83	1.00	1.00	0.29	0.54	0.88	0.92
Fixed-bucket-I(1)	0.83	1.00	1.00	1.00	0.79	0.92	1.00	1.00	0.71	0.92	1.00	1.00
Variable-bucket-I(5,10)	0.83	1.00	1.00	1.00	0.71	0.83	1.00	1.00	0.71	0.92	1.00	1.00

optimal solution, the optimizer is stopped and the best feasible solution found is provided. Table 2 shows the computational times (in seconds) required to solve all instances to optimality. The computational time required tends to increase when the cardinality m of the portfolio decreases. A symbol (*) indicates that the solver failed to solve the instance within 12 hours. This happened for 4 out of 24 instances.

In the following we provide and comment only the main results of the extensive computational analysis performed. Results reported in Tables 3 and 4 compare the solutions found by the heuristics with the optimal ones. In the cases where the optimal solution was not found within the time limit, the best solution obtained has been used instead. Interesting enough, on these instances, heuristics *Shifted-bucket*, *Variable-bucket*(5,10), *Fixed-bucket*(1) and *Fixed-bucket*(0.2) with Sorting A or Sorting B obtain results up to 0.5% better than the solutions obtained by CPLEX in 12 hours.

Table 3 provides the percentage of solved instances with an error, with respect to the optimal solution, lower than or equal to the value γ reported in the columns. The

Author's personal copy

Table 4 Integer optimal solutions: rank of items and bucket length to guarantee optimality

	Rank of items selected by IP-Model(N) Sorting **A**/B				Minimum bucket length Sorting **A**/B			
	$\beta = 1\%$ $m = 10$	$\beta = 5\%$ $m = 10$	$\beta = 1\%$ $m = 20$	$\beta = 5\%$ $m = 20$	$\beta = 1\%$ $m = 10$	$\beta = 5\%$ $m = 10$	$\beta = 1\%$ $m = 20$	$\beta = 5\%$ $m = 20$
100	**11**/11	**23**/45	**13**/13	**15**/15	**0**/0	**1**/2	**0**/0	**0**/0
200	**21**/21	**25**/25	**41**/71	**38**/63	**0**/0	**0**/0	**1**/2	**1**/2
300	**56**/95	**56**/95	**61**/141	**61**/141	**1**/2	**1**/2	**1**/4	**1**/4
400	**38**/71	**171**/88	**44**/88	**44**/88	**1**/2	**5**/2	**1**/2	**1**/2
500	**12**/12	**68**/74	**110**/241	**110**/241	**0**/0	**1**/2	**3**/6	**3**/6
600	**90**/183	**217**/227	**117**/238	**117**/238	**2**/4	**5**/6	**3**/6	**3**/6

table is divided into three parts, each of which provides the findings for a different sorting rule. For heuristic *Fixed-bucket*(0,0), whose final kernel corresponds to the items selected in the optimal solution of the continuous relaxation problem, the type of sorting is irrelevant. For all other heuristics, Sorting A dominates on average Sorting B which, in turn, outperforms Sorting R. Only for heuristic *Fixed-bucket*(0.2) and its iterative version *Fixed-bucket-I*(0.2) the percentage of instances solved to optimality is higher for Sorting B with respect to Sorting A. Similarly, for heuristic *Fixed-bucket*(0.2) with $\gamma = 1\%$. The result was somehow expected as Sorting B considers general properties of the linear relaxation while Sorting A exploits specific features of the problem. However, the difference in terms of performance between Sorting B and Sorting R is much more remarkable than that between Sorting A and Sorting B.

In Table 3 the basic variants are listed first (lines 1–9) according to their average solution quality obtained with Sorting A, while some of their iterative variants are listed second (last 5 lines). There are three basic heuristics (see lines 7, 8, 9) that, with Sorting A, find the optimal solution in 79% of cases (20 instances out of 24) and in all the instances within 1% from the optimum.

When we compare the performance of a basic variant (lines 2, 3, 5, 7 and 8) with the corresponding iterative variant we can see that, in general, the latter provides a better performance. However, the larger improvements are obtained for those heuristics with poor performance (lines 2, 3 and 5). In this case, however, the improved performances are never better than those obtained by the basic variant of the best heuristics (lines 7–9). On the other hand, the iterative variants of good basic heuristics (lines 7, 8) improve only a little upon the basic variant (1 instance out of 24) while such small achievement is obtained at the cost of a much higher computational time. The number of iterations is in most cases equal to 2 and never greater than 3.

When we focus on the heuristics taking into account a limited number of buckets (lines 2, 4, 5 and 6 of Table 3) it can be noticed that, by applying the right sorting, it is possible to obtain good solutions. In general, the quality of the solutions tends to increase with the length of the buckets. A larger number of buckets seems to prevent relations among items and thus provide worse results. Take, for instance, heuristics *Fixed-bucket*(0.2,10), *Fixed-bucket*(1,2) and *Fixed-bucket*(2,1). All these three heuristics consider a total number of $3C$ items; the first C items are automatically included

Author's personal copy

in the initial kernel and the other $2C$ items are divided into buckets. These last $2C$ items are divided into 10 buckets of length $\frac{1}{5}C$, into 2 buckets of length C, and finally into 1 bucket only of length $2C$, respectively. Comparing results in Table 3, it is evident that a lower number of buckets provides better results for all types of implemented sorting. On the other hand, larger buckets imply, on average, larger computational times.

To further compare the effectiveness of Sorting A with respect to Sorting B, in Table 4 we have compared, for each instance, the maximum rank obtained by items in the optimal integer solution according to the two types of sorting. Moreover, we report the minimum bucket length k such that heuristic *Fixed-bucket*(k,1) can obtain the optimal integer solution. In the first group of columns, we report the maximum position held by an item selected by the optimal solution when ranked according to Sorting A (figures in bold) and when ranked according to Sorting B. For instance, values **23**/45 in the first row and second column of the table mean that, once the items selected in the optimal solution of the continuous relaxation problem have been sorted according to Sorting A (B), the highest position held in such sorting by an item selected by the integer optimal solution was 23 (45). In the second group of columns, we report the minimum value of parameter k required for the two types of sorting in order to obtain an optimal solution by heuristic *Fixed-bucket*(k,1) that considers only one bucket beside the initial kernel. For instance, values **3**/6 in row 5 and column 7 mean that heuristic *Fixed-bucket*(3,1) would obtain the optimal integer solution if applied with Sorting A, while using Sorting B the optimal solution would require to run *Fixed-bucket*(6,1). Null values, like in row 1 and column 5, mean that the instance is solved to the optimum by considering just the items in the initial kernel. This is the case, for instance, for heuristic *Fixed-bucket*(0,0). We recall that results for $m = 10$ and $n = 500, 600$ are not reliable since we do not have a guaranteed optimal solution for such instances. On the contrary, we have guarantee of optimality for the instances with $n = 500, 600$ when $m = 20$. In all cases the computational time of heuristic *Fixed-bucket*(k,1) increases rapidly with k.

Figures 2 and 3 show the average (over the two values 1% and 5% of the quantile size β) computational times of the heuristics with Sorting A, that have shown the best performance out of all solved instances. Heuristic *Shifted-bucket* analyzes the whole set of items almost completely twice. However, its good solution performance is not better than that of other heuristics like *Fixed-bucket*(1) and *Variable-bucket*(5,10), and the computational time is much higher. Even for $k = 2$ the computational times of heuristic *Fixed-bucket*(k,1) are very high with respect to other heuristics that can guarantee similar or better results. As seen in Table 4, its performance could be improved by increasing the bucket length, but the computational time would increase accordingly. On the contrary, we note that *Fixed-bucket*(1,2) is not only extremely efficient but also quite effective. Heuristic *Fixed-bucket*(1) is the extension of *Fixed-bucket*(1,2) to all the possible buckets. We can see from Table 3 that it gains in solution performance without a dramatic increase in computational times. Nevertheless, its main drawback is to spend too much time, especially on the last buckets, considering sets of buckets that are unlikely to add more than one item to the kernel. Heuristic *Variable-bucket*(5,10) tries to solve the trade-off between size and number of problems. It considers more problems with smaller size with respect to *Fixed-bucket*(1).

Author's personal copy

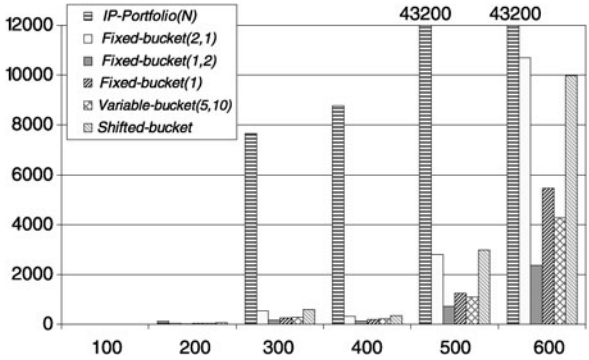


Fig. 2 Average (over $\beta = 1\%$ and $\beta = 5\%$) computational times (in seconds) for all levels of capital: $m = 10$ and Sorting A

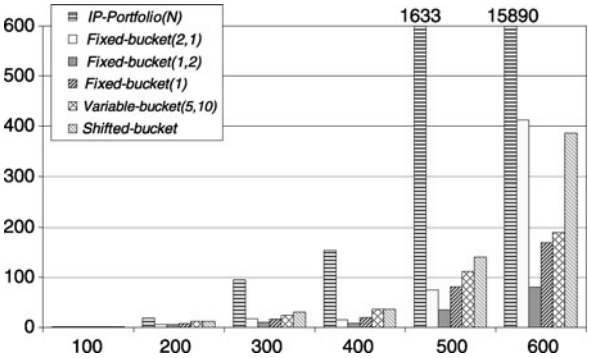


Fig. 3 Average (over $\beta = 1\%$ and $\beta = 5\%$) computational times (in seconds) for all levels of capital: $m = 20$ and Sorting A

For $m = 10$ such heuristic seems to give good results, while for $m = 20$ heuristic *Fixed-bucket*(1) is faster. In the worst case ($m = 10$, $n = 600$), heuristic *Variable-bucket*(5,10) takes 4279 seconds while *Fixed-bucket*(1) takes 5454 seconds.

5.2 American data set

In order to evaluate whether the performance of the proposed heuristics is affected by a change of data, we carried out computational results on a different data set consisting of 400 stocks from the American Stock Exchange. We consider 104 historical realizations representing weekly rates of return over the period 2000–2001. The rates are computed as relative stock price variations and dividends are not included. We have considered the same commissions (fixed and proportional) used for the European Stock Exchanges and the same levels of capital C_0 and C_1. The tolerance level β and the yearly required return have been set to 5%. Different instances have been created according to the number of securities used ($n = 100, 200, 300, 400$ securities, respectively) and the threshold on portfolio cardinality ($m = 10$ and 20). This means 8 instances altogether.

We have decided to test one among the heuristics which performed best on the previous data set and its iterative variant, i.e. *Fixed-bucket*(1) and *Fixed-bucket*-I(1), with sorting criterion A. In the following we only show the results provided by CPLEX and heuristic *Fixed-bucket*(1) since its iterative variant always finds the same solution at a slightly greater computational time.

Author's personal copy

Table 5 Experimental results: $m = 10$

n	CPLEX				*Fixed-bucket*(1)			
	time	objective	best bound	gap	time	objective	CPLEX-gap	bound-gap
100	135.19	−2117.90	−2117.90	0.00%	45.81	−2117.90	0.00%	0.00%
200	33369.30	−1766.10	−1766.10	0.00%	2490.92	−1766.10	0.00%	0.00%
300	43200.03	−1687.84	−1283.89	31.46%	12015.03	−1586.52	−6.00%	23.57%
400	43200.05	−1596.40	−1104.81	44.50%	16354.92	−1506.69	−5.62%	36.38%

Table 6 Experimental results: $m = 20$

n	CPLEX		*Fixed-bucket*(1)		
	time	objective	time	objective	CPLEX-gap
100	34.23	−2450.99	13.77	−2462.78	0.48%
200	971.89	−1796.30	105.20	−1808.22	0.66%
300	2272.72	−1593.84	161.06	−1660.92	4.21%
400	36379.45	−1479.08	719.83	−1526.19	3.18%

In Table 5 the total computational time (in seconds), the objective function values of both the solution found by CPLEX and by heuristic *Fixed-bucket*(1), the best bound available and the percentage gap found from this bound (bound-gap) and from CPLEX (CPLEX-gap) are shown. The table refers to the case in which the cardinality constraint has $m = 10$. The same results for $m = 20$ are shown in Table 6. In the latter case no best bound is shown as it is always equal to the optimal solution value found by CPLEX.

Instances with cardinality threshold set to 10 are more difficult, as shown in Table 5, where CPLEX finds the optimal solution in two minutes for $n = 100$, but requires more than 9 hours for $n = 200$ and is unable to find an optimal solution within 12 hours when $n \geq 300$. With this cardinality, the computational time required by *Fixed-bucket*(1) also increases but the performance improves as well since the heuristic finds the optimal solution for $n = 200$ and is very close to it for $n = 100$, requiring, respectively, almost 1/14 and 1/3 of the computational time required by CPLEX. In the remaining instances the results provided by the heuristic are consistently better than those found by CPLEX. The improvement is, on average, equal to 5.81% and has been obtained with less than 1/3 of the computational time.

In Table 6 CPLEX is able to find the optimal solution of the instance with $n = 100$ in few seconds, but requires more than 10 hours when $n = 400$. Heuristic *Fixed-bucket*(1,1) performs always well requiring less than 12 minutes and providing solutions with an average error equal to 2.13%.

5.3 Summary of the experimental analysis

The analysis of all the results has led us to draw the following main conclusions:

1. The securities selected by the continuous relaxation of the MILP model are a good initial kernel.

Author's personal copy

2. Sorting the securities in order of weighted reduced costs of the continuous relaxation of the original MILP problem is a good sorting criterion to consider securities to add to the initial kernel.
3. The quality of the solution provided by a heuristic increases with the length of the buckets, while its efficiency decreases with the buckets size (larger buckets imply larger computational times).
4. The basic Heuristics *Fixed-bucket*(1) and *Variable-bucket*(5,10) show the best trade-off between quality of the solution and computational time.

Note that the Kernel Search framework can be applied to any other portfolio optimization problem by simply using as *IP-Model*(.) the MILP model that defines the problem. The diversification effect guarantees that the number of securities in the initial kernel is such that the MILP model restricted to the initial kernel, and to the successive kernels, can be easily solved by a commercial software.

Conclusions and future research

In this paper we have introduced a new heuristic framework called Kernel Search for the solution of portfolio optimization problems with real features. We have analyzed a specific problem, that is modeled as a mixed integer linear programming (MILP) problem, but the Kernel Search can be applied to any other portfolio optimization problem, provided that the number of selected securities in the optimal solution is not too large. The Kernel Search requires the solution of the continuous relaxation of the MILP problem and uses the information provided by its optimal solution (values of reduced costs) to sort the securities. The ordered set is then divided into an initial kernel and a sequence of groups (buckets) of securities. These buckets are iteratively analyzed through the solution of MILP problems restricted to a kernel of securities in order to extend, at each iteration, the previous kernel. The diversification effect guarantees that the number of securities selected by the continuous relaxation is not too large and that the restricted MILPs can be easily solved by a commercial software. We have provided a detailed computational analysis finalized to the identification, in the Kernel Search, of the best specific heuristic. The obtained results are extremely good both in terms of quality of the solutions and of computational time required. The high quality of the solutions found mostly relies on the exact solution of the MILP problems restricted to promising sets of securities, i.e. on the capability to deeply explore good regions of the solution space.

Future research should be devoted to prove that the Kernel Search is a valuable approach for its robustness. The same heuristic framework can be easily extended to other combinatorial problems characterized by the presence of items to be selected.

Acknowledgements The authors wish to acknowledge the useful comments and suggestions of two anonymous reviewers who helped them improve a previous version of this paper.

References

1. Angelelli, E., Mansini, R., Speranza, M.G.: A comparison of MAD and CVaR models with real features. J. Bank. Finance **32**, 1188–1197 (2008)

Author's personal copy

2. Archetti, C., Speranza, M.G., Savelsbergh, M.: An optimization-based heuristic for the split delivery vehicle routing problem. Transp. Sci. **42**, 22–31 (2008)
3. Balas, E., Zemel, E.: An algorithm for large zero-one knapsack problems. Oper. Res. **28**, 1130–1154 (1980)
4. Chang, T.-J., Meade, N., Beasley, J.E., Sharaiha, Y.M.: Heuristics for cardinality constrained portfolio optimisation. Comput. Oper. Res. **27**, 1271–1302 (2000)
5. Chiodi, L., Mansini, R., Speranza, M.G.: Semi-absolute deviation rule for mutual funds portfolio selection. Ann. Oper. Res. **124**, 245–265 (2003)
6. De Franceschi, R., Fischetti, M., Toth, P.: A new ILP-based refinement heuristic for vehicle routing problems. Math. Program. **105**, 471–499 (2006)
7. Guastaroba, G., Mansini, R., Speranza, M.G.: On the effectiveness of scenario generation techniques in single-period portfolio optimization. Eur. J. Oper. Res. **192**, 500–511 (2009)
8. Jobst, N.J., Horniman, M.H., Lucas, C., Mitra, G.: Computational aspects of alternative portfolio selection models in the presence of discrete asset choice constraints. Quant. Finance **1**, 489–501 (2001)
9. Kellerer, H., Mansini, R., Speranza, M.G.: Selecting portfolios with fixed costs and minimum transaction lots. Ann. Oper. Res. **99**, 287–304 (2000)
10. Konno, H., Wijayanayake, A.: Portfolio optimization problem under concave transaction costs and minimal transaction unit constraints. Math. Program. **89**, 233–250 (2001)
11. Konno, H., Yamamoto, R.: Integer programming approaches in mean-risk models. Comput. Manag. Sci. **2**, 339–351 (2005)
12. Konno, H., Yamamoto, R.: Applications of integer programming to financial optimization. In: Zopounidis, et al. (eds.), Handbook of Financial Engineering, pp. 25–48. Springer, Berlin (2008)
13. Mansini, R., Speranza, M.G.: Heuristic algorithms for the portfolio selection problem with minimum transaction lots. Eur. J. Oper. Res. **114**, 219–233 (1999)
14. Mansini, R., Speranza, M.G.: An exact approach for the portfolio selection problem with transaction costs and rounds. IIE Trans. **37**, 919–929 (2005)
15. Mansini, R., Ogryczak, W., Speranza, M.G.: LP Solvable models for portfolio optimization: A classification and computational comparison. IMA J. Manag. Math. **14**, 187–220 (2003)
16. Markowitz, H.M.: Portfolio selection. J. Finance **7**, 77–91 (1952)
17. Markowitz, H.M.: Portfolio Selection: Efficient Diversification of Investments. Wiley, New York (1959)
18. Pisinger, D.: Core problems in knapsack algorithms. Oper. Res. **47**, 570–575 (1999)